SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Technical Presentation
Weightage: 20%

QUESTIONS: **Total Marks: 20**

1. **Servlet Architecture & Workflow**

Question:
Present the **architecture of Java Servlets** and explain the complete **request–response cycle** with a neat diagram.

Focus Areas:

- Client–Server interaction
- Role of Web Container
- Request and Response objects

Java Servlet Architecture

A Java Servlet is a server-side Java program that processes client requests and generates responses. The main components are the **Client, Web Server, Servlet Container, Servlet, Request Object, and Response Object**.

Request–Response Cycle

Client / Browser
↓
HTTP Request
↓
Web Server
↓
Servlet Container
↓
Servlet

↓
Process Request
↓
Generate Response
↓
Servlet Container
↓
HTTP Response
↓
Client / Browser

Explanation

1. The client sends an HTTP request to the web server.
2. The web server forwards the request to the **Servlet Container**.
3. The Servlet Container identifies and invokes the appropriate Servlet.
4. The Servlet processes the request using the **HttpServletRequest** object.
5. The Servlet creates a response using **HttpServletResponse**.
6. The response is sent back to the client.

The **Web Container** manages Servlet creation, execution, lifecycle, and communication between the client and Servlet.

---

2. Servlet Life Cycle Analysis

Question:

Explain the **Servlet Life Cycle** with a sequence diagram and analyze the role of init(), service(), and destroy() methods.

Focus Areas:

- Life cycle phases
- Execution flow
- Performance implications

2. Servlet Life Cycle Analysis

The Servlet life cycle consists mainly of **initialization, request processing, and destruction**.

Sequence Diagram

```
Servlet Container
     |
     | Create Servlet
     ↓
   init()
     |
     | Initialization completed
     ↓
  service()
     |
     |——→ doGet()
     |
```

```
     └──→ doPost()
     |
     | Multiple requests
     ↓
  service()
     |
     | Server shuts down / Servlet removed
     ↓
  destroy()
     |
     ↓
Servlet Removed
```

init()

- Called **once** when the Servlet is initialized.
- Used for initialization tasks.
- Resources required by the Servlet can be prepared here.

service()

- Called whenever the Servlet receives a request.
- Processes client requests.
- Depending on the HTTP method, it invokes methods such as doGet() or doPost().

destroy()

- Called when the Servlet is being removed.
- Used to release resources and perform cleanup.

Performance

Because init() is normally called only once, initialization work does not need to be repeated for every request. The service() method handles multiple requests, while destroy() ensures resources are properly released.

---

3. GET vs POST Method Evaluation

Question:

Compare **GET and POST methods** used in form submission. Present scenarios where each method is appropriate.

Focus Areas:

- Data transmission
- Security aspects
- URL limitations
- GET and POST are HTTP methods commonly used for submitting data to a Servlet.

Feature	GET	POST
Data transmission	Data is appended to URL	Data is sent in request body
Visibility	Data visible in URL	Data not displayed in URL

Feature	GET	POST
URL limitation	Limited by URL length	Supports larger amounts of data
Security	Less suitable for sensitive data	More appropriate for sensitive data when combined with HTTPS
Typical use	Retrieving/searching information	Submitting or creating information

- GET Example
- `/student?name=Sairaj&regno=192411018`
- GET is appropriate when retrieving information or performing searches where the parameters can safely appear in the URL.

- POST Example
- `Student Registration Form`
- `↓`
- `POST`
- `↓`
- `StudentServlet`
- POST is appropriate for registration forms, login forms, and other operations where data should not be exposed in the URL.
- **Important:** POST by itself does not encrypt data. Sensitive information should be transmitted using **HTTPS**.

---

4. Session Tracking Techniques
Question:
Present different **session tracking mechanisms** (HttpSession, Cookies, URL rewriting). Evaluate their advantages and limitations.
Focus Areas:

- Session persistence
- Security concerns
- Use-case comparison

HTTP is stateless, so web applications need session tracking mechanisms to identify and maintain information about a user across multiple requests.
The three techniques required are **HttpSession, Cookies, and URL Rewriting**.
1. HttpSession
HttpSession stores session information on the server.
HttpSession session = request.getSession();
session.setAttribute("username", "Sairaj");
Advantages:

- Session information is maintained on the server.
- Suitable for maintaining login information.
- Can be invalidated when the user logs out.

Limitation:

- Requires server-side memory/resources.

---

2. Cookies

Cookies store small pieces of information in the user's browser.

```
Cookie cookie = new Cookie("username", "Sairaj");
response.addCookie(cookie);
```

Advantages:

- Simple to implement.
- Can persist information between visits.
- Useful for preferences and remembering users.

Limitations:

- Stored on the client side.
- Users can disable or delete cookies.
- Sensitive information should not be stored directly in cookies.

---

3. URL Rewriting

The session information is added to the URL.

https://example.com/home;jsessionid=12345

Advantages:

- Can work when cookies are disabled.
- Does not depend on browser cookie storage.

Limitations:

- Session information can appear in the URL.
- URLs become longer.
- Can create security concerns if sensitive information is exposed.

---

5. Cookies vs Sessions – Critical Analysis

Question:

Analyze and compare **Cookies and Sessions** in web applications. Present real-time use cases.

Focus Areas:

- Storage location
- Security risks
- Performance considerations

5. Cookies vs Sessions – Critical Analysis

Feature	Cookies	Sessions
Storage	Client/browser	Server
Data size	Small	Can maintain more server-side information
Security	More exposed to client-side attacks/misuse	Generally better for sensitive session state

Feature	Cookies	Sessions
Persistence	Can persist after browser closes depending on configuration	Usually associated with a session lifetime
Performance	Reduces some server-side storage requirements	Requires server resources
Common use	Preferences, remembering settings	Login/authentication state

Real-Time Examples

Cookies:

- Remembering language preference.
- Storing website preferences.
- Remembering that a user has selected certain settings.

Sessions:

- Maintaining a logged-in user's identity.
- Shopping cart information.
- Maintaining authentication across multiple pages.

Conclusion

Cookies are useful when information needs to be stored on the client, while **sessions are more suitable for maintaining user-specific information on the server**, particularly for authenticated web applications. The choice depends on storage location, security requirements, persistence, and performance.

Code of Conduct:

I <u>Sairaj C(192411018)</u> (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: <u>Sairaj C</u>

RUBRICS FOR CALCULATION:

Technical Presentation					
Criteria	**Weight age**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Technical Content Accuracy	30	Highly accurate, in-depth, and insightful technical explanation	Technically correct content with adequate depth and clarity	Basic concepts presented with limited accuracy and depth	Content contains major technical errors; concepts poorly understood
Problem Identification	25	Problem rigorously analyzed using appropriate and advanced methods	Problem clearly defined with a logical engineering approach	Problem identified but analysis or methodology is weak	Problem statement unclear or incorrect; no logical approach
Use of Tools	20	Advanced tools, well-analyzed data, and highly effective visuals	Appropriate tools and data used effectively with clear visuals	Limited use of tools and basic data interpretation	Minimal or incorrect use of tools and data; poor visuals
Communication	15	Highly engaging, confident, and audience-focused delivery	Clear, organized, and professional communication	Understandable but lacks clarity, structure, or confidence	Poor organization, unclear speech, ineffective slides
Professionalism	10	Exemplary professionalism, leadership, and ethical awareness	Professional conduct with effective team collaboration	Basic professionalism; uneven team participation	Lacks professionalism; minimal contribution or ethical awareness